

LeetCode 300 — Longest Increasing Subsequence (LIS)

BINARY SEARCH

DP

PATIENCE SORT

Comprehensive Technical Documentation & Algorithmic Approaches

Problem

Given an array `nums`, find the length of the longest strictly increasing subsequence.

Subsequence → Elements remain in the same order, but you may skip elements.

Strictly Increasing → Every next element must be greater than the previous one.

```
// Example
nums = [10, 9, 2, 5, 3, 7, 101, 18] | LIS = [2, 3, 7, 101] | Length = 4
```

Approach 1 — Binary Search (Patience Sorting) ★★★★★

INTUITION

Instead of storing the actual LIS, we only store the best possible ending value for every subsequence length.

IDEA

A smaller ending element is always better because it has a greater chance of being extended later.

Example: Length 1 ending options:

- Ending = 8
- Ending = 3 ← **Better**

Because $3 < 8$. Later numbers like 4, 5, 6 can extend 3, but cannot extend 8. So we always try to keep the ending as small as possible.

WHAT DOES TAILS[I] MEAN?

`tails[i]` = Smallest ending value of an Increasing Subsequence of length `(i+1)`.

tails Index	0	1	2
Value	2	3	7

means: Length 1 → ends with 2 | Length 2 → ends with 3 | Length 3 → ends with 7

These are not necessarily one subsequence. They are simply the best endings.

WORKING EXAMPLE

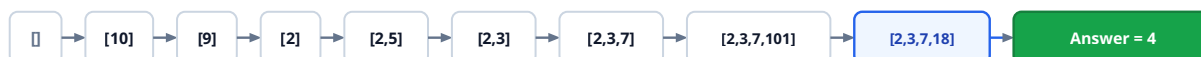
nums: 10 9 2 5 3 7 101 18

Step	Current Element	Action	tails State	Why / Explanation
Step 1	10	Insert 10	[10]	First element.
Step 2	Current = 9	Find first ≥ 9 (10 ↑) → Replace	[9]	Ending 9 is better than Ending 10.
Step 3	Current = 2	Find first ≥ 2 (9 ↑) → Replace	[2]	Smaller ending.
Step 4	Current = 5	$5 > 2$ → Append	[2, 5]	Length 1 ending = 2, Length 2 ending = 5.
Step 5	Current = 3	Find first ≥ 3 (2 5 ↑) → Replace	[2, 3]	2 3 is better than 2 5 because Ending 3 can be extended more easily.
Step 6	Current = 7	$7 > 3$ → Append	[2, 3, 7]	Extends LIS length.
Step 7	Current = 101	$101 > 7$ → Append	[2, 3, 7, 101]	Extends LIS length.
Step 8	Current = 18	Find first ≥ 18 (101 ↑) → Replace	[2, 3, 7, 18]	Ending 18 is better than 101.

Final tails size = 4 ⇒ **Answer = 4**

STEP PROGRESSION DIAGRAM

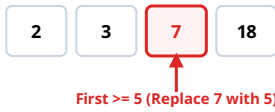
Input: 10 9 2 5 3 7 101 18



Approach 1 — Implementation & Complexity

WHY BINARY SEARCH?

Suppose `tails: 2 3 7 18`. Need to insert `5`. We need the first element `>= 5`.



Instead of scanning linearly $O(n)$, **Binary Search** finds it in $O(\log n)$ because `tails` is always sorted.

CODE FLOW (C++)

```
vector<int> tails; // Stores best ending values

for(int x : nums) { // Process every element
    auto it = lower_bound(tails.begin(), tails.end())
                // Find first element >= x

    if(it == tails.end()) {
        // Case 1: x is larger than every element
        tails.push_back(x); // LIS length increases
    } else {
        // Case 2: Replace old ending
        *it = x; // Length same, ending smaller
    }
}

return tails.size(); // Final LIS length
```

For every element

→

Binary Search

→

$O(\log n)$

|

Total Time: $O(n \log n)$

|

Space: $O(n)$

Approach 2 — Bottom-Up DP

INTUITION

For every index `i`, assume the LIS ends at `i`. Look at all previous indices.

If `nums[j] < nums[i]` then `nums[i]` can extend the subsequence ending at `j`.

DP STATE & TRANSITION

`dp[i]` = Length of LIS ending at `i`

```
dp[i] = 1 + max(dp[j]) where j < i and
nums[j] < nums[i]
```

CODE FLOW (C++)

```
vector<int> dp(n, 1); // LIS alone = 1

for(int i = 0; i < n; i++) { // Treat i as ending index
    for(int j = 0; j < i; j++) { // Check previous
        if(nums[j] < nums[i]) { // Can extend
            dp[i] = max(dp[i], dp[j] + 1);
        }
    }
    ans = max(ans, dp[i]); // Overall maximum
}
```

EXAMPLE WALKTHROUGH

`nums`: 10 9 2 5 3 7

Initially `dp`: 1 1 1 1 1 1

• For 5: $2 < 5 \rightarrow$ `dp`: 1 1 1 **2** 1 1

• For 3: $2 < 3 \rightarrow$ `dp`: 1 1 1 2 **2** 1

• For 7: Possible previous: 2, 5, 3. Best `dp = 2` $\rightarrow 2 + 1 = 3 \rightarrow$

`dp`: 1 1 1 2 2 **3**

Answer = $\max(dp) = 3$

COMPLEXITY

Time Complexity $O(n^2)$ — Nested loop scans previous elements

Space Complexity $O(n)$ — 1D array for DP states

Approach 3 — Memoization

INTUITION

Same DP as Approach 2, but computed recursively.

Instead of filling left to right, recursively ask:

"What is the LIS ending at every previous index?"

STATE & RECURRENCE

`solve(i)` = LIS ending at `i`

```
solve(i) = 1 + max(solve(j)) where j < i and
nums[j] < nums[i]
```

MEMOIZATION OPTIMIZATION

`dp[i]` = Already computed answer $\rightarrow$ Return directly $\rightarrow$ Avoid recomputation.

COMPLEXITY

Time Complexity $O(n^2)$

Space Complexity $O(n)$

Approach 4 — Take / Not Take DP

INTUITION

At every index, make a decision: **Take** or **Skip**. Exactly like Knapsack.

STATE

`solve(i, prev)`

- `i` = Current index
- `prev` = Last chosen index

TRANSITIONS

Skip: `solve(i+1, prev)`

Take: Only if `prev == -1` or `nums[i] > nums[prev]` → `1 + solve(i+1, i)`

Take maximum of both options.

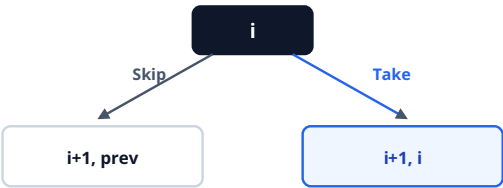
DP TABLE MAPPING & INDEX OFFSET

DP Table: `dp[i][prev+1]`

Why +1? Because `prev` can be `-1`. Array index cannot be negative.

Time Complexity $O(n^2)$

DECISION TREE DIAGRAM



prev	-1	0	1	2	...
Mapped Index	0	1	2	3	...

Space Complexity $O(n^2)$

Final Comparison Summary

Approach	Core Idea	Time	Space
Binary Search	Store smallest possible ending value	$O(n \log n)$	$O(n)$
Bottom-Up DP	LIS ending at each index	$O(n^2)$	$O(n)$
Memoization	Recursive version of DP	$O(n^2)$	$O(n)$
Take / Not Take	Choose take/skip at every index	$O(n^2)$	$O(n^2)$



Interview Tip:

Use the **Binary Search** approach when only the length of the LIS is required. Use the **DP approaches** when the problem involves reconstructing the sequence or adding extra constraints.